Project Definition Document

IndustryApp – post-accident repair offer platform · PA1

Authors: Ömer Efe Öztürk, Bahadır İhsan Herdem · **Course:** BIL 482 Design Patterns · **Version:** 1.0 · **Date:** 05/10/2026

Table of Contents

1. Document Task Matrix
2. Project Name
3. Project Summary
4. Objectives
5. Scope
6. Target Audience
7. Key Features
8. Deliverables
9. Project Success Criteria

1. Document Task Matrix

#	Task in creating this document	Responsible	Reviewer
T1	Project name and summary	Bahadır İhsan Herdem	Ömer Efe Öztürk
T2	Objectives	Ömer Efe Öztürk	Bahadır İhsan Herdem
T3	Scope (included / not included)	Bahadır İhsan Herdem	Ömer Efe Öztürk
T4	Target audience	Ömer Efe Öztürk	Bahadır İhsan Herdem
T5	Key features	Ömer Efe Öztürk	Bahadır İhsan Herdem
T6	Deliverables	Bahadır İhsan Herdem	Ömer Efe Öztürk

#	Task in creating this document	Responsible	Reviewer
T7	Project success criteria	Bahadır İhsan Herdem	Ömer Efe Öztürk
T8	Final review, formatting, GitHub upload	Ömer Efe Öztürk	Bahadır İhsan Herdem

2. Project Name

IndustryApp – Post-Accident Repair Offer Platform. "Industry" refers to the *sanayi* districts where independent body, paint and mechanical workshops operate.

3. Project Summary

IndustryApp is a reverse-auction marketplace that connects vehicle owners who have had an accident with repair workshops (mechanics). The customer reports the damage step by step with photos, damaged areas, vehicle type and location. The system then finds suitable, approved mechanics through an event-driven matching process and notifies them instantly. Mechanics submit sealed offers with a price breakdown, the customer compares and selects one, and both sides' contact details are revealed only at that moment.

The problem it solves is the opacity of post-accident repair: owners phone workshops one by one, cannot compare prices fairly, and expose their phone number to everyone they ask. IndustryApp turns this into one structured request, comparable sealed offers and privacy-preserving contact sharing. It is designed to run in production from day one: no demo data, real users, HTTPS and daily backups.

4. Objectives

By the end of the project the team will have:

1. Implemented four core use cases end to end: role-based registration and login (UC-01), accident report creation (UC-02), job notification and offer submission (UC-03), and offer selection with contact reveal (UC-04).
2. Built a layered, modular monolith (Clean Architecture, .NET 8 Web API + React/TypeScript) in which modules communicate through an in-process publish–subscribe event bus.
3. Applied at least one GoF or architectural design pattern per use case — 16 patterns in 18 places — each traceable by a `PAT` code from document to code comment to test.

4. Delivered real-time (SignalR) and e-mail notifications that keep working when the e-mail provider fails (Circuit Breaker + Retry).
5. Protected personal data: contact details only for the matched pair, EXIF/GPS removed from photos, KVKK consent stored with document version.
6. Deployed the system to a public domain over HTTPS with health checks and nightly backups, and run the final demo there.
7. Produced the PA1–PA3 documents and a test report with full UC → FR → PAT → TC traceability.

5. Scope

Included in this project:

1. Role-based registration and login for customers and mechanics, with mandatory acceptance of role-specific legal documents; mechanic accounts require admin approval.
2. Step-by-step accident report creation: 1–8 photos, damaged areas from a three-level damage tree with severity, vehicle type, location.
3. Automatic matching of suitable mechanics when a report is created, and notification of the matched mechanics.
4. Sealed offers by mechanics with a price breakdown (labor, parts, paint, warranty, other); update and withdraw while pending.
5. Live offer list for the customer, sorting by price, estimated duration or newest, and selection of one offer.
6. Mutual contact reveal after selection, undo of the selection within a configurable window (default 10 minutes), completion and cancellation.
7. In-app (real-time) and e-mail notifications.
8. Administration: mechanic approval and damage category management.
9. Production deployment on a single Linux server with HTTPS, health check and nightly backups.

Not included (Future Work):

- Payment and commission
- Ratings and reviews
- In-app messaging between customer and mechanic
- SMS provider integration (the channel interface exists; no provider is connected)
- Native mobile application (the API is mobile-ready)
- Insurance company integration

- Map-based mechanic search

6. Target Audience

- **Customers:** individual vehicle owners who have had an accident and need body, paint, mechanical, electrical, glass or tyre repair. They are everyday smartphone and web users with no technical background.
- **Mechanics:** small and medium repair workshops in industrial districts (body, paint, mechanical and similar trades) who want a steady flow of jobs without price-undercutting. Digital skills vary, so screens must stay simple.
- **Administrators:** the platform operator, who approves mechanic accounts and maintains the damage catalog.

7. Key Features

1. **Role-based accounts:** separate sign-up forms for customer and mechanic, legal consent, admin approval for mechanics, and a role-specific dashboard after login.
2. **Accident report wizard:** five steps (photos → damage areas → vehicle → location → summary); incomplete reports cannot be created.
3. **Photo processing:** every upload is resized, compressed and stripped of EXIF/GPS metadata before it is stored.
4. **Hierarchical damage catalog:** a three-level tree (e.g. Body → Front → Bumper) that admins extend without code changes.
5. **Automatic mechanic matching:** city- or radius-based candidate search, then filters for approval, specialty, availability and daily capacity.
6. **Sealed offers with price breakdown:** each mechanic sees only their own offer; the total is computed from labor, parts, paint, warranty and other items.
7. **Live comparison and selection:** offers appear without a page refresh and can be sorted by price, estimated days or newest; one offer is selected with confirmation.
8. **Undo within a time window:** the customer can reverse a selection for 10 minutes, with a visible countdown.
9. **Protected contact reveal:** name, phone and e-mail are shown only to the customer and the selected mechanic; everyone else sees a masked number.
10. **Notifications:** real-time in-app notifications and e-mail; e-mail failures do not block in-app delivery.
11. **Automatic expiry:** a report without a selection closes after 72 hours and the customer is informed.

12. **Admin panel:** mechanic approval or rejection with a reason, and damage category management.

8. Deliverables

1. **PA1 package** (this assignment): Project Definition, Project Plan, Software Requirements Specification, Software Architecture Overview, Pattern–Use Case Mapping Matrix.
2. **PA2 package:** Software Design Document for UC-04 (class, sequence, component and ER diagrams, UI screenshots), source code tagged `v0.2`, `README.md`, `TASK_MATRIX.md`.
3. **PA3 package:** Software Design Document updated for all four use cases (incl. deployment diagram, change log, Future Work), test report `TestReport.xlsx`, source code tagged `v1.0`.
4. **Running production system** on a public domain over HTTPS, used for the final demo.
5. **GitHub repository** with documents, source code, database scripts and deployment files, with per-member commit history.

9. Project Success Criteria

1. All four use cases work end to end in the production environment with real users, and all 8 end-to-end scenarios (E2E-01 … E2E-08) pass.
2. The report creation request responds in under 500 ms (p95), independent of matching and notification time.
3. No unmatched user can see the other party's real phone number in any API response (verified by planned tests TC-04.4.1 and E2E-07).
4. A report can never hold two accepted offers at the same time; this is guaranteed by a database constraint (planned test TC-04.2.3).
5. All 15 automated tests are green; test coverage is at least 70% for Domain and Application, and at least 90% for pattern classes.
6. In-app notifications keep working when the e-mail provider is down (E2E-06).
7. Every use case has at least one design pattern whose `PAT` code appears in the code comment, the design document and a test.